



UNIVERSIDAD NACIONAL DE INGENIERÍA
FACULTAD DE INGENIERÍA ELÉCTRICA Y ELECTRÓNICA
Escuela de Ingeniería en Ciberseguridad

SGISI

Sistema de Gestión de Incidentes de
Seguridad Informática

Informe Técnico del Proyecto

Curso: Base de Datos I (CBS04M)
Docente: Carpio Farfán, Pedro Manuel
Integrantes: Borja Soto, Diego
Escajadillo Gaspar, José
Valencia Casanova, Fabrizio

Lima, Perú – 2026

Índice

1. Justificación	2
2. Diagrama	2
3. Esquema lógico	4
4. Esquema físico	5
5. Programas	7
5.1. Funciones	7
5.2. Procedimientos almacenados	9
5.3. Triggers	11
5.4. Cursor	13
6. C# y Web	14
6.1. Aplicación de escritorio en C#	14
6.2. Página web	16
7. Hardening	17
7.1. Menor privilegio: logins nominales	17
7.2. Enmascaramiento Dinámico de Datos (DDM)	18
7.3. Auditoría de accesos	18

1. Justificación

Una empresa peruana de consultoría en ciberseguridad brinda servicios de monitoreo y respuesta a varias organizaciones. Debido a que anteriormente la información se registraba en hojas de cálculo compartidas, se generaban problemas de pérdida de información, duplicidad de registros y falta de trazabilidad sobre los incidentes detectados y los activos comprometidos.

Por ello, se desarrolló un **Sistema de Gestión de Incidentes de Seguridad Informática (SGISI)** que permite almacenar de manera centralizada la información de las organizaciones, registrando para cada una un identificador único, el nombre del departamento responsable, el responsable del área y la cantidad de activos administrados. Cada organización puede contar con varios analistas de seguridad y administrar múltiples activos tecnológicos, mientras que cada analista y cada activo pertenecen a una única organización.

De los analistas se almacena un identificador, nombre, apellido, cargo, especialización, nivel de acceso, correo electrónico y teléfono; mientras que de los activos se registra un identificador, nombre, dirección IP, sistema operativo y nivel de criticidad. Adicionalmente, los analistas se clasifican en dos tipos: Supervisores y Supervisados. Un analista supervisor puede supervisar a varios analistas supervisados, mientras que cada analista supervisado depende de un único supervisor; del supervisor se registra el número de analistas a su cargo, y del supervisado su nivel de experiencia y el área en la que opera. Un analista solo puede pertenecer a uno de estos tipos, por lo que no puede ser supervisor y supervisado al mismo tiempo.

Los activos pueden presentar diversas vulnerabilidades de seguridad, registrándose para cada uno un identificador, código CVE, descripción, severidad CVSS, fecha de descubrimiento y estado. Los analistas son responsables de gestionar los incidentes de seguridad: un analista puede atender varios incidentes, mientras que cada incidente tiene un único analista responsable. Los activos generan alertas de seguridad; varias alertas pueden escalar a un mismo incidente, aunque una alerta puede no estar asociada a ninguno. Un incidente puede afectar a varios activos y un mismo activo puede verse comprometido en distintos incidentes, registrándose para cada asociación el impacto ocasionado y la fecha de detección. Finalmente, los incidentes pueden generar uno o varios reportes, elaborados por un analista.

Con el fin de garantizar la trazabilidad de las operaciones efectuadas sobre los incidentes, el sistema mantiene además una **bitácora de auditoría** que conserva el historial de todos los cambios realizados sobre ellos, asociada al analista responsable del incidente auditado.

2. Diagrama

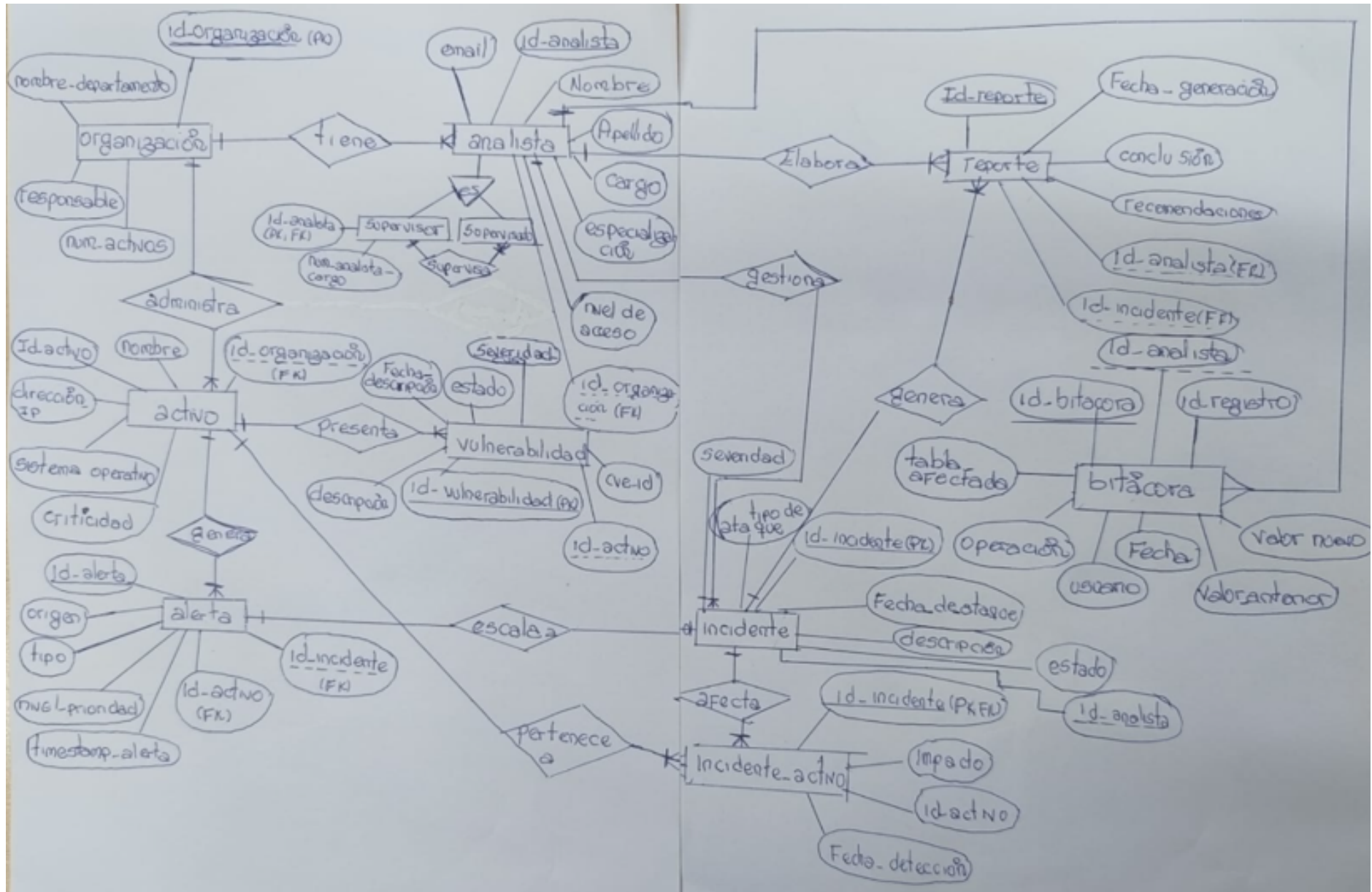


Figura 1: Diagrama entidad-relación de SGISI.

3. Esquema lógico

A continuación se presenta el esquema lógico relacional de SGISI: cada entidad se expresa como TABLA(atributos), indicando su clave primaria (PK) y sus claves foráneas (FK).

ORGANIZACION(id_organizacion, nombre_departamento, responsable, num_activos)

PK = id_organizacion

ANALISTA(id_analista, nombre, apellido, cargo, especializacion, nivel_acceso, email, telefono, id_organizacion)

PK = id_analista

FK = id_organizacion de ORGANIZACION

SUPERVISOR(id_analista, num_analistas_cargo)

PK = id_analista

FK = id_analista de ANALISTA

SUPERVISADO(id_analista, nivel_experiencia, area_operacion)

PK = id_analista

FK = id_analista de ANALISTA

ACTIVO(id_activo, nombre, direccion_ip, sistema_operativo, criticidad, id_organizacion)

PK = id_activo

FK = id_organizacion de ORGANIZACION

VULNERABILIDAD(id_vulnerabilidad, cve_id, descripcion, severidad_cvss, fecha_descubrimiento, estado, id_activo)

PK = id_vulnerabilidad

FK = id_activo de ACTIVO

INCIDENTE(id_incidente, tipo_ataque, fecha_hora, descripcion, estado, severidad, id_analista)

PK = id_incidente

FK = id_analista de ANALISTA

ALERTA(id_alerta, origen, timestamp_alerta, tipo, nivel_prioridad, id_activo, id_incidente)

PK = id_alerta

FK = id_activo de ACTIVO

FK = id_incidente de INCIDENTE (opcional)

INCIDENTE_ACTIVO(id_incidente, id_activo, impacto, fecha_deteccion)

PK = (id_incidente, id_activo)

FK = id_incidente de INCIDENTE

FK = id_activo de ACTIVO

REPORTE(id_reporte, fecha_generacion, conclusiones, recomendaciones, id_analista, id_incidente)

PK = id_reporte

FK = id_analista de ANALISTA

FK = id_incidente de INCIDENTE

BITACORA(id_bitacora, tabla_afectada, operacion, id_registro, usuario, fecha, valor_anterior, valor_nuevo, id_analista)

PK = id_bitacora

FK = id_analista de ANALISTA

(id_registro NO es FK a INCIDENTE de forma deliberada, para no perder el rastro de auditoría si el incidente llegara a eliminarse.)

4. Esquema físico

Se presenta a continuación el código T-SQL de creación de las 11 tablas de SGISI, con sus atributos, tipos de datos y restricciones (claves primarias, claves foráneas, CHECK, DEFAULT y UNIQUE).

```

1 CREATE DATABASE SGISI;
2 GO
3 USE SGISI;
4 GO
5
6 CREATE TABLE ORGANIZACION (
7     id_organizacion INT IDENTITY(1,1) PRIMARY KEY,
8     nombre_departamento VARCHAR(100) NOT NULL,
9     responsable VARCHAR(100) NOT NULL,
10    num_activos INT NULL DEFAULT (0)
11    CHECK (num_activos >= 0)
12 );
13 GO
14
15 CREATE TABLE ANALISTA (
16     id_analista INT IDENTITY(1,1) PRIMARY KEY,
17     nombre VARCHAR(80) NOT NULL,
18     apellido VARCHAR(80) NOT NULL,
19     cargo VARCHAR(100) NOT NULL,
20     especializacion VARCHAR(100) NULL,
21     nivel_acceso TINYINT NOT NULL
22     CHECK (nivel_acceso >= 1 AND nivel_acceso <= 5),
23     email VARCHAR(150) NOT NULL UNIQUE,
24     telefono VARCHAR(20) NULL,
25     id_organizacion INT NOT NULL,
26     CONSTRAINT FK_ANALISTA_ORG FOREIGN KEY (id_organizacion)
27     REFERENCES ORGANIZACION(id_organizacion) ON DELETE CASCADE
28 );
29 GO
30
31 CREATE TABLE SUPERVISOR (
32     id_analista INT PRIMARY KEY,
33     num_analistas_cargo INT NULL DEFAULT (0)
34     CHECK (num_analistas_cargo >= 0),
35     CONSTRAINT FK_SUPERVISOR_ANALISTA FOREIGN KEY (id_analista)
36     REFERENCES ANALISTA(id_analista) ON DELETE CASCADE
37 );
38 GO
39
40 CREATE TABLE SUPERVISADO (
41     id_analista INT PRIMARY KEY,
42     nivel_experiencia VARCHAR(20) NOT NULL
43     CHECK (nivel_experiencia IN ('Senior','Mid','Junior')),
44     area_operacion VARCHAR(100) NOT NULL,
45     CONSTRAINT FK_SUPERVISADO_ANALISTA FOREIGN KEY (id_analista)
46     REFERENCES ANALISTA(id_analista) ON DELETE CASCADE
47 );
48 GO
49
50 CREATE TABLE ACTIVO (
51     id_activo INT IDENTITY(1,1) PRIMARY KEY,
52     nombre VARCHAR(100) NOT NULL,
53     direccion_ip VARCHAR(45) NULL,
54     sistema_operativo VARCHAR(80) NULL,
55     criticidad VARCHAR(10) NOT NULL
56     CHECK (criticidad IN ('CRITICA','ALTA','MEDIA','BAJA')),
57     id_organizacion INT NOT NULL,
58     CONSTRAINT FK_ACTIVO_ORG FOREIGN KEY (id_organizacion)
59     REFERENCES ORGANIZACION(id_organizacion)
60 );
61 GO
62
63 CREATE TABLE VULNERABILIDAD (
64     id_vulnerabilidad INT IDENTITY(1,1) PRIMARY KEY,
65     cve_id VARCHAR(30) NULL UNIQUE,
66     descripcion VARCHAR(500) NOT NULL,

```

```

67     severidad_cvss          DECIMAL(3,1) NULL
68                             CHECK (severidad_cvss >= 0.0 AND severidad_cvss <= 10.0),
69     fecha_descubrimiento    DATE NOT NULL DEFAULT (GETDATE()),
70     estado                  VARCHAR(20) NOT NULL
71                             CHECK (estado IN ('PARCHEADA','EN_PROCESO','PENDIENTE')),
72     id_activo               INT NOT NULL,
73     CONSTRAINT FK_VULN_ACTIVO FOREIGN KEY (id_activo)
74         REFERENCES ACTIVO(id_activo) ON DELETE CASCADE
75 );
76 GO
77
78 CREATE TABLE INCIDENTE (
79     id_incidente INT IDENTITY(1,1) PRIMARY KEY,
80     tipo_ataque  VARCHAR(100) NOT NULL,
81     fecha_hora   DATETIME NOT NULL DEFAULT (GETDATE()),
82     descripcion  VARCHAR(1000) NULL,
83     estado       VARCHAR(20) NOT NULL
84                 CHECK (estado IN ('ABIERTO','EN_INVESTIGACION','CERRADO')),
85     severidad    DECIMAL(3,1) NULL
86                 CHECK (severidad >= 0.0 AND severidad <= 10.0),
87     id_analista  INT NOT NULL,
88     CONSTRAINT FK_INCIDENTE_ANALISTA FOREIGN KEY (id_analista)
89         REFERENCES ANALISTA(id_analista)
90 );
91 GO
92
93 CREATE TABLE ALERTA (
94     id_alerta      INT IDENTITY(1,1) PRIMARY KEY,
95     origen         VARCHAR(50) NOT NULL
96                 CHECK (origen IN ('MANUAL','ANTIVIRUS','FIREWALL','SIEM','IDS')),
97     timestamp_alerta DATETIME NOT NULL DEFAULT (GETDATE()),
98     tipo           VARCHAR(100) NOT NULL,
99     nivel_prioridad VARCHAR(10) NOT NULL
100                 CHECK (nivel_prioridad IN ('CRITICA','ALTA','MEDIA','BAJA')),
101     id_activo      INT NOT NULL,
102     id_incidente   INT NULL,
103     CONSTRAINT FK_ALERTA_ACTIVO FOREIGN KEY (id_activo)
104         REFERENCES ACTIVO(id_activo) ON DELETE CASCADE,
105     CONSTRAINT FK_ALERTA_INCIDENTE FOREIGN KEY (id_incidente)
106         REFERENCES INCIDENTE(id_incidente)
107 );
108 GO
109
110 CREATE TABLE INCIDENTE_ACTIVO (
111     id_incidente INT NOT NULL,
112     id_activo    INT NOT NULL,
113     impacto      VARCHAR(20) NULL
114                 CHECK (impacto IN ('CRITICO','GRAVE','MODERADO','LEVE')),
115     fecha_deteccion DATETIME NULL DEFAULT (GETDATE()),
116     CONSTRAINT PK_INCIDENTE_ACTIVO PRIMARY KEY (id_incidente, id_activo),
117     CONSTRAINT FK_IA_INCIDENTE FOREIGN KEY (id_incidente)
118         REFERENCES INCIDENTE(id_incidente) ON DELETE CASCADE,
119     CONSTRAINT FK_IA_ACTIVO FOREIGN KEY (id_activo)
120         REFERENCES ACTIVO(id_activo) ON DELETE CASCADE
121 );
122 GO
123
124 CREATE TABLE REPORTE (
125     id_reporte      INT IDENTITY(1,1) PRIMARY KEY,
126     fecha_generacion DATE NOT NULL DEFAULT (GETDATE()),
127     conclusiones     VARCHAR(2000) NULL,
128     recomendaciones  VARCHAR(2000) NULL,
129     id_analista      INT NOT NULL,
130     id_incidente     INT NOT NULL,
131     CONSTRAINT FK_REPORTER_ANALISTA FOREIGN KEY (id_analista)
132         REFERENCES ANALISTA(id_analista),
133     CONSTRAINT FK_REPORTER_INCIDENTE FOREIGN KEY (id_incidente)
134         REFERENCES INCIDENTE(id_incidente) ON DELETE CASCADE
135 );
136 GO
137
138 -- Entidad de auditoria: registra los cambios sobre INCIDENTE
139 CREATE TABLE BITACORA (

```

```
140 id_bitacora INT IDENTITY(1,1) PRIMARY KEY,  
141 tabla_afectada VARCHAR(100) NOT NULL,  
142 operacion VARCHAR(10) NOT NULL  
143 CHECK (operacion IN ('INSERT','UPDATE','DELETE')),  
144 id_registro INT NULL,  
145 usuario SYSNAME NOT NULL DEFAULT (USER_SNAME()),  
146 fecha DATETIME NOT NULL DEFAULT (GETDATE()),  
147 valor_anterior VARCHAR(MAX) NULL,  
148 valor_nuevo VARCHAR(MAX) NULL,  
149 id_analista INT NULL,  
150 CONSTRAINT FK_BITACORA_ANALISTA FOREIGN KEY (id_analista)  
151 REFERENCES ANALISTA(id_analista)  
152 );  
153 GO
```

5. Programas

Además de las estructuras estáticas de datos, SGISI implementa lógica de negocio directamente en el motor de la base de datos mediante cuatro tipos de objetos programables: funciones, procedimientos almacenados, triggers y un cursor.

5.1. Funciones

Una **función** es un objeto que recibe parámetros y devuelve un valor escalar o una tabla; se invoca dentro de consultas y *no modifica* los datos. SGISI utiliza diez funciones:

fn_BuscarOrganizacion

Busca una organización por su identificador.

fn_BuscarAnalista

Busca un analista por su identificador.

fn_BuscarIncidente

Busca incidentes por identificador o por coincidencia parcial en el tipo de ataque.

fn_BuscarAlerta

Busca alertas por identificador o por coincidencia parcial en el tipo.

fn_BuscarReporte

Busca reportes por identificador o por coincidencia parcial en las conclusiones.

fn_BuscarVulnerabilidad

Busca una vulnerabilidad por su identificador.

fn_HistorialIncidentesPorActivo

Devuelve todos los incidentes que han afectado a un activo específico, junto con el impacto ocasionado.

fn_ListarActivosCriticos

Lista los activos de criticidad ALTA o CRÍTICA de una organización.

fn_ContarIncidentesPorAnalista

Devuelve cuántos incidentes tiene asignados un analista.

fn_SeveridadPromedioOrganizacion

Calcula la severidad promedio de los incidentes de una organización.


```
1 CREATE FUNCTION fn_BuscarOrganizacion (@id INT = NULL, @nombre_depto VARCHAR(100) = NULL)
2 RETURNS TABLE AS RETURN
3 (
4     SELECT id_organizacion, num_activos
5     FROM dbo.ORGANIZACION
6     WHERE (@id IS NOT NULL AND id_organizacion = @id)
7 )
8 GO
9
10 CREATE FUNCTION fn_BuscarAnalista (@id INT = NULL, @cargo VARCHAR(100) = NULL)
11 RETURNS TABLE AS RETURN
12 (
13     SELECT id_analista, id_organizacion
14     FROM dbo.ANALISTA
15     WHERE (@id IS NOT NULL AND id_analista = @id)
16 )
17 GO
18
19 CREATE FUNCTION fn_BuscarIncidente (@id INT = NULL, @tipo_ataque VARCHAR(100) = NULL)
20 RETURNS TABLE AS RETURN
21 (
22     SELECT id_incidente, tipo_ataque, fecha_hora, descripcion, estado, severidad, id_analista
23     FROM dbo.INCIDENTE
24     WHERE (@id IS NOT NULL AND id_incidente = @id)
25           OR (@tipo_ataque IS NOT NULL AND tipo_ataque LIKE '%' + @tipo_ataque + '%')
26 );
27 GO
28
29 CREATE FUNCTION fn_BuscarAlerta (@id INT = NULL, @tipo VARCHAR(100) = NULL)
30 RETURNS TABLE AS RETURN
31 (
32     SELECT id_alerta, origen, timestamp_alerta, tipo, nivel_prioridad, id_activo, id_incidente
33     FROM dbo.ALERTA
34     WHERE (@id IS NOT NULL AND id_alerta = @id)
35           OR (@tipo IS NOT NULL AND tipo LIKE '%' + @tipo + '%')
36 );
37 GO
38
39 CREATE FUNCTION fn_BuscarReporte (@id INT = NULL, @conclusiones VARCHAR(2000) = NULL)
40 RETURNS TABLE AS RETURN
41 (
42     SELECT id_reporte, fecha_generacion, conclusiones, recomendaciones, id_incidente, id_analista
43     FROM dbo.REPORTE
44     WHERE (@id IS NOT NULL AND id_reporte = @id)
45           OR (@conclusiones IS NOT NULL AND conclusiones LIKE '%' + @conclusiones + '%')
46 );
47 GO
48
49 CREATE FUNCTION fn_BuscarVulnerabilidad (@id INT = NULL)
50 RETURNS TABLE AS RETURN
51 (
52     SELECT id_vulnerabilidad, descripcion, estado
53     FROM dbo.VULNERABILIDAD
54     WHERE (@id IS NOT NULL AND id_vulnerabilidad = @id)
55 );
56 GO
57
58 CREATE FUNCTION fn_HistorialIncidentesPorActivo (@id_activo INT)
59 RETURNS TABLE
60 AS
61 RETURN
62 (
63     SELECT
64         IA.id_activo,
65         I.id_incidente,
66         I.tipo_ataque,
67         I.estado AS estado_incidente,
68         I.severidad AS severidad_incidente,
69         IA.impacto,
70         IA.fecha_deteccion
71     FROM dbo.INCIDENTE_ACTIVO IA
72     JOIN dbo.INCIDENTE I ON IA.id_incidente = I.id_incidente
73     WHERE IA.id_activo = @id_activo
```

```

74 );
75 GO
76
77 CREATE FUNCTION fn_ListarActivosCriticos(@id_organizacion INT)
78 RETURNS TABLE
79 AS
80 RETURN
81 (
82     SELECT id_activo, nombre, direccion_ip, sistema_operativo, criticidad
83     FROM ACTIVO
84     WHERE id_organizacion = @id_organizacion
85           AND criticidad IN ('ALTA','CRITICA')
86 );
87 GO
88
89 CREATE FUNCTION fn_ContarIncidentesPorAnalista(@id_analista INT)
90 RETURNS INT
91 AS
92 BEGIN
93     DECLARE @total INT;
94     SELECT @total = COUNT(*)
95     FROM INCIDENTE
96     WHERE id_analista = @id_analista;
97     RETURN @total;
98 END;
99 GO
100
101 CREATE FUNCTION fn_SeveridadPromedioOrganizacion(@id_organizacion INT)
102 RETURNS DECIMAL(5,2)
103 AS
104 BEGIN
105     DECLARE @promedio DECIMAL(5,2);
106     SELECT @promedio = AVG(I.severidad)
107     FROM INCIDENTE I
108     JOIN ANALISTA A ON I.id_analista = A.id_analista
109     WHERE A.id_organizacion = @id_organizacion;
110     RETURN ISNULL(@promedio, 0);
111 END;
112 GO

```

5.2. Procedimientos almacenados

Un **procedimiento almacenado** es un bloque de instrucciones que sí ejecuta acciones sobre los datos (insertar, actualizar); en SGISI, cada uno de escritura se protege con una **transacción** explícita (BEGIN TRANSACTION / COMMIT / ROLLBACK) dentro de un bloque TRY...CATCH, de modo que ante cualquier error toda la operación se deshace por completo. SGISI utiliza seis procedimientos:

sp_RegistrarIncidente

Registra un nuevo incidente, asignándole la fecha exacta y el estado inicial ABIERTO.

sp_RegistrarAlerta

Registra una alerta generada por un activo.

sp_AsignarActivoAlIncidente

Vincula un activo a un incidente en la tabla puente, validando que ambos existan.

sp_CerrarIncidente

Cambia el estado del incidente a CERRADO y, en la misma transacción, registra el reporte con conclusiones y recomendaciones.

sp_GenerarReporte

Genera un reporte para un incidente existente.

sp_ResumenSeguridadPorAnalista

Procedimiento con **cursor** (se explica en la sección 5.4).

```
1 CREATE PROCEDURE sp_RegistrarIncidente
2     @tipo_ataque VARCHAR(100),
3     @descripcion VARCHAR(1000),
4     @severidad DECIMAL(3,1),
5     @id_analista INT
6 AS
7 BEGIN
8     SET NOCOUNT ON;
9     BEGIN TRANSACTION;
10    BEGIN TRY
11        INSERT INTO INCIDENTE
12            (tipo_ataque, fecha_hora, descripcion, estado, severidad, id_analista)
13        VALUES
14            (@tipo_ataque, GETDATE(), @descripcion, 'ABIERTO', @severidad, @id_analista);
15        COMMIT TRANSACTION;
16        SELECT SCOPE_IDENTITY() AS id_incidente_creado;
17    END TRY
18    BEGIN CATCH
19        ROLLBACK TRANSACTION;
20        THROW;
21    END CATCH
22 END;
23 GO
24
25 CREATE PROCEDURE sp_RegistrarAlerta
26     @origen VARCHAR(50),
27     @tipo VARCHAR(100),
28     @nivel_prioridad VARCHAR(10),
29     @id_activo INT,
30     @id_incidente INT = NULL
31 AS
32 BEGIN
33     BEGIN TRANSACTION;
34     BEGIN TRY
35        INSERT INTO ALERTA
36            (origen, timestamp_alerta, tipo, nivel_prioridad, id_activo, id_incidente)
37        VALUES
38            (@origen, GETDATE(), @tipo, @nivel_prioridad, @id_activo, @id_incidente);
39        COMMIT TRANSACTION;
40        SELECT SCOPE_IDENTITY() AS id_alerta_creada;
41    END TRY
42    BEGIN CATCH
43        ROLLBACK TRANSACTION;
44        THROW;
45    END CATCH
46 END;
47 GO
48
49 CREATE PROCEDURE sp_AsignarActivoAIncidente
50     @id_incidente INT,
51     @id_activo INT,
52     @impacto VARCHAR(20)
53 AS
54 BEGIN
55     BEGIN TRANSACTION;
56     BEGIN TRY
57        IF NOT EXISTS (SELECT 1 FROM INCIDENTE WHERE id_incidente = @id_incidente)
58            THROW 50001, 'El incidente especificado no existe.', 1;
59        IF NOT EXISTS (SELECT 1 FROM ACTIVO WHERE id_activo = @id_activo)
60            THROW 50002, 'El activo especificado no existe.', 1;
61        INSERT INTO INCIDENTE_ACTIVO (id_incidente, id_activo, impacto, fecha_deteccion)
62        VALUES (@id_incidente, @id_activo, @impacto, GETDATE());
63        COMMIT TRANSACTION;
64    END TRY
65    BEGIN CATCH
66        ROLLBACK TRANSACTION;
67        THROW;
68    END CATCH
69 END;
```

```

70 GO
71
72 CREATE PROCEDURE sp_CerrarIncidente
73     @id_incidente INT,
74     @conclusiones VARCHAR(2000),
75     @recomendaciones VARCHAR(2000),
76     @id_analista INT
77 AS
78 BEGIN
79     BEGIN TRANSACTION;
80     BEGIN TRY
81         UPDATE INCIDENTE
82         SET estado = 'CERRADO'
83         WHERE id_incidente = @id_incidente;
84
85         INSERT INTO REPORTE
86         (fecha_generacion, conclusiones, recomendaciones, id_analista, id_incidente)
87         VALUES
88         (GETDATE(), @conclusiones, @recomendaciones, @id_analista, @id_incidente);
89     COMMIT TRANSACTION;
90     END TRY
91     BEGIN CATCH
92         ROLLBACK TRANSACTION;
93         THROW;
94     END CATCH
95 END;
96 GO
97
98 CREATE PROCEDURE sp_GenerarReporte
99     @id_incidente INT,
100     @conclusiones VARCHAR(2000),
101     @recomendaciones VARCHAR(2000),
102     @id_analista INT
103 AS
104 BEGIN
105     BEGIN TRANSACTION;
106     BEGIN TRY
107         IF NOT EXISTS (SELECT 1 FROM INCIDENTE WHERE id_incidente = @id_incidente)
108             THROW 50003, 'El incidente especificado no existe.', 1;
109         INSERT INTO REPORTE
110         (fecha_generacion, conclusiones, recomendaciones, id_analista, id_incidente)
111         VALUES
112         (GETDATE(), @conclusiones, @recomendaciones, @id_analista, @id_incidente);
113     COMMIT TRANSACTION;
114     SELECT SCOPE_IDENTITY() AS id_reporte_creado;
115     END TRY
116     BEGIN CATCH
117         ROLLBACK TRANSACTION;
118         THROW;
119     END CATCH
120 END;
121 GO

```

5.3. Triggers

Un **trigger** (disparador) es un objeto que se ejecuta automáticamente como reacción a un cambio en una tabla (INSERT, UPDATE o DELETE), sin que ninguna aplicación lo invoque explícitamente. SGISI utiliza seis triggers:

trg_ActualizarNumActivos_Insert

Incrementa el contador de activos de la organización cuando se agrega un activo.

trg_ActualizarNumActivos_Delete

Decrementa el contador de activos cuando se elimina un activo.

trg_ActualizarNumActivos_Update

Ajusta los contadores de origen y destino cuando un activo cambia de organización.

trg_ValidarEspecializacionDisjunta

Impide registrar como *Supervisado* a un analista que ya es *Supervisor*.

trg_ValidarSupervisorDisjunto

Impide registrar como *Supervisor* a un analista que ya es *Supervisado*.

trg_AuditarIncidente

Ante cualquier INSERT, UPDATE o DELETE sobre INCIDENTE, registra el cambio en BITACORA (usuario, fecha, valores anterior y nuevo), sin intervención de la aplicación.

```
1 CREATE TRIGGER trg_ActualizarNumActivos_Insert
2 ON ACTIVO
3 AFTER INSERT
4 AS
5 BEGIN
6     UPDATE ORGANIZACION
7     SET num_activos = num_activos + 1
8     WHERE id_organizacion IN (SELECT id_organizacion FROM inserted);
9 END;
10 GO
11
12 CREATE TRIGGER trg_ActualizarNumActivos_Delete
13 ON ACTIVO
14 AFTER DELETE
15 AS
16 BEGIN
17     UPDATE ORGANIZACION
18     SET num_activos = num_activos - 1
19     WHERE id_organizacion IN (SELECT id_organizacion FROM deleted);
20 END;
21 GO
22
23 CREATE TRIGGER trg_ActualizarNumActivos_Update
24 ON dbo.ACTIVO
25 AFTER UPDATE
26 AS
27 BEGIN
28     SET NOCOUNT ON;
29     IF UPDATE(id_organizacion)
30     BEGIN
31         UPDATE o
32         SET o.num_activos = o.num_activos - x.cnt
33         FROM dbo.ORGANIZACION o
34         JOIN (SELECT id_organizacion, COUNT(*) AS cnt
35              FROM deleted GROUP BY id_organizacion) x
36              ON o.id_organizacion = x.id_organizacion;
37
38         UPDATE o
39         SET o.num_activos = o.num_activos + x.cnt
40         FROM dbo.ORGANIZACION o
41         JOIN (SELECT id_organizacion, COUNT(*) AS cnt
42              FROM inserted GROUP BY id_organizacion) x
43              ON o.id_organizacion = x.id_organizacion;
44     END
45 END;
46 GO
47
48 CREATE TRIGGER trg_ValidarEspecializacionDisjunta
49 ON SUPERVISADO
50 AFTER INSERT
51 AS
52 BEGIN
53     IF EXISTS (
54         SELECT 1 FROM inserted I
55         JOIN SUPERVISOR S ON I.id_analista = S.id_analista
56     )
57     BEGIN
58         ROLLBACK TRANSACTION;
59         THROW 50010, 'Un analista no puede ser Supervisor y Supervisado al mismo tiempo.', 1;
60     END
```

```

61 END;
62 GO
63
64 CREATE TRIGGER trg_ValidarSupervisorDisjunto
65 ON SUPERVISOR
66 AFTER INSERT
67 AS
68 BEGIN
69     IF EXISTS (
70         SELECT 1 FROM inserted I
71         JOIN SUPERVISADO S ON I.id_analista = S.id_analista
72     )
73     BEGIN
74         ROLLBACK TRANSACTION;
75         THROW 50011, 'Un analista no puede ser Supervisado y Supervisor al mismo tiempo.', 1;
76     END
77 END;
78 GO
79
80 CREATE TRIGGER trg_AuditarIncidente
81 ON dbo.INCIDENTE
82 AFTER INSERT, UPDATE, DELETE
83 AS
84 BEGIN
85     SET NOCOUNT ON;
86     DECLARE @hayIns BIT = CASE WHEN EXISTS(SELECT 1 FROM inserted) THEN 1 ELSE 0 END;
87     DECLARE @hayDel BIT = CASE WHEN EXISTS(SELECT 1 FROM deleted) THEN 1 ELSE 0 END;
88
89     IF @hayIns = 1 AND @hayDel = 1
90     BEGIN
91         INSERT INTO dbo.BITACORA (tabla_afectada, operacion, id_registro, id_analista, valor_anterior,
92             valor_nuevo)
93         SELECT 'INCIDENTE', 'UPDATE', i.id_incidente, i.id_analista,
94             'estado=' + ISNULL(d.estado COLLATE DATABASE_DEFAULT, 'NULL')
95             + '; severidad=' + ISNULL(CAST(d.severidad AS VARCHAR(10)), 'NULL')
96             + '; analista=' + ISNULL(CAST(d.id_analista AS VARCHAR(10)), 'NULL'),
97             'estado=' + ISNULL(i.estado COLLATE DATABASE_DEFAULT, 'NULL')
98             + '; severidad=' + ISNULL(CAST(i.severidad AS VARCHAR(10)), 'NULL')
99             + '; analista=' + ISNULL(CAST(i.id_analista AS VARCHAR(10)), 'NULL')
100         FROM inserted i JOIN deleted d ON i.id_incidente = d.id_incidente;
101     END
102     ELSE IF @hayIns = 1
103     BEGIN
104         INSERT INTO dbo.BITACORA (tabla_afectada, operacion, id_registro, id_analista, valor_anterior,
105             valor_nuevo)
106         SELECT 'INCIDENTE', 'INSERT', i.id_incidente, i.id_analista, NULL,
107             'estado=' + ISNULL(i.estado COLLATE DATABASE_DEFAULT, 'NULL')
108             + '; severidad=' + ISNULL(CAST(i.severidad AS VARCHAR(10)), 'NULL')
109             + '; tipo=' + ISNULL(i.tipo_ataque COLLATE DATABASE_DEFAULT, 'NULL')
110         FROM inserted i;
111     END
112     ELSE IF @hayDel = 1
113     BEGIN
114         INSERT INTO dbo.BITACORA (tabla_afectada, operacion, id_registro, id_analista, valor_anterior,
115             valor_nuevo)
116         SELECT 'INCIDENTE', 'DELETE', d.id_incidente, d.id_analista,
117             'estado=' + ISNULL(d.estado COLLATE DATABASE_DEFAULT, 'NULL')
118             + '; severidad=' + ISNULL(CAST(d.severidad AS VARCHAR(10)), 'NULL')
119             + '; tipo=' + ISNULL(d.tipo_ataque COLLATE DATABASE_DEFAULT, 'NULL'),
120             NULL
121         FROM deleted d;
122     END
123 END;
124 GO

```

5.4. Cursor

Un **cursor** permite recorrer un resultado *fila por fila* de manera secuencial, a diferencia de las operaciones habituales que procesan un conjunto completo de registros a la vez. En SGISI, el procedimiento `sp_ResumenSeguridadPorAnalista` utiliza un cursor para recorrer, uno por uno, a to-

dos los analistas registrados y, para cada uno, invoca las funciones `fn_ContarIncidentesPorAnalista` y `fn-SeveridadPromedioOrganizacion`, consolidando el resultado en un único reporte de resumen.

```

1 CREATE PROCEDURE dbo.sp_ResumenSeguridadPorAnalista
2 AS
3 BEGIN
4     SET NOCOUNT ON;
5
6     DECLARE @id INT, @org INT, @total INT, @prom DECIMAL(5,2);
7
8     -- Tabla temporal para devolver el resumen a la aplicacion (DataGridView en C#)
9     DECLARE @Resumen TABLE (
10         id_analista INT,
11         id_organizacion INT,
12         total_incidentes INT,
13         severidad_prom_org DECIMAL(5,2)
14     );
15
16     -- 1) DECLARE
17     DECLARE cur_analistas CURSOR LOCAL FAST_FORWARD FOR
18         SELECT id_analista, id_organizacion
19         FROM dbo.ANALISTA
20         ORDER BY id_analista;
21
22     -- 2) OPEN
23     OPEN cur_analistas;
24
25     -- 3) Primer FETCH
26     FETCH NEXT FROM cur_analistas INTO @id, @org;
27
28     -- 4) Recorrido fila por fila
29     WHILE @@FETCH_STATUS = 0
30     BEGIN
31         SET @total = dbo.fn_ContarIncidentesPorAnalista(@id);
32         SET @prom = dbo.fn-SeveridadPromedioOrganizacion(@org);
33
34         PRINT 'Analista ' + CAST(@id AS VARCHAR(10))
35             + ' | Incidentes: ' + CAST(@total AS VARCHAR(10))
36             + ' | Severidad prom. org: ' + CAST(@prom AS VARCHAR(10));
37
38         INSERT INTO @Resumen (id_analista, id_organizacion, total_incidentes, severidad_prom_org)
39         VALUES (@id, @org, @total, @prom);
40
41         FETCH NEXT FROM cur_analistas INTO @id, @org;
42     END
43
44     -- 5) CLOSE y DEALLOCATE
45     CLOSE cur_analistas;
46     DEALLOCATE cur_analistas;
47
48     -- Resultado para la aplicacion
49     SELECT id_analista, id_organizacion, total_incidentes, severidad_prom_org
50     FROM @Resumen
51     ORDER BY total_incidentes DESC;
52 END;
53 GO

```

6. C# y Web

6.1. Aplicación de escritorio en C#

El profesor solicitó explícitamente una aplicación cliente construida en **C# con Visual Studio** que se conectara a la base de datos que él proporcionaría, con el fin de demostrar el uso de un cursor y, posteriormente, del hardening implementado. Se optó por **Windows Forms**, por ser el tipo de proyecto más simple para construir una interfaz de escritorio con botones y una grilla de resultados (`DataGridView`), usando `Microsoft.Data.SqlClient` (`ADO.NET`) para conectarse a SQL Server. En vez de definir usuarios ficticios dentro del código, la aplicación exige

un **login real de SQL Server**: al conectarse, se le pregunta directamente al motor si ese login es administrador (`db_owner` o con el permiso `UNMASK`) o un cliente de solo consulta, y la interfaz se adapta en consecuencia (ocultando los botones de escritura para los clientes).

```

1 // Conexion.cs - mantiene la conexion ACTIVA de la sesion actual
2 public static class Conexion
3 {
4     private const string Servidor = @"localhost\SQLEXPRESS";
5     private const string BaseDatos = "SGISI";
6
7     public static string CadenaConexion { get; private set; } = "";
8     public static string NombreUsuario { get; private set; } = "";
9     public static bool EsAdministrador { get; private set; }
10
11     public static string ConstruirCadena(string usuario, string contrasena)
12     {
13         SqlConnectionStringBuilder builder = new SqlConnectionStringBuilder
14         {
15             DataSource = Servidor,
16             InitialCatalog = BaseDatos,
17             UserID = usuario,
18             Password = contrasena,
19             TrustServerCertificate = true
20         };
21         return builder.ConnectionString;
22     }
23
24     public static void IniciarSesion(string usuario, string contrasena, bool esAdministrador)
25     {
26         CadenaConexion = ConstruirCadena(usuario, contrasena);
27         NombreUsuario = usuario;
28         EsAdministrador = esAdministrador;
29     }
30
31     public static SqlConnection Crear() => new SqlConnection(CadenaConexion);
32 }

```

```

1 // FormLogin.cs - pantalla de inicio de sesion (evento del boton Ingresar)
2 private void btnIngresar_Click(object? sender, EventArgs e)
3 {
4     string usuario = txtUsuario.Text.Trim();
5     string contrasena = txtContrasena.Text;
6     string cadena = Conexion.ConstruirCadena(usuario, contrasena);
7
8     using SqlConnection cn = new SqlConnection(cadena);
9     cn.Open();
10
11     // Le preguntamos a SQL Server si este login es administrador
12     // (db_owner o tiene el permiso UNMASK), en vez de adivinarlo en C#.
13     using SqlCommand cmd = new SqlCommand(
14         "SELECT CASE " +
15         "  WHEN IS_ROLEMEMBER('db_owner')= 1 THEN 1 " +
16         "  WHEN EXISTS (SELECT 1 FROM sys.database_permissions dp " +
17         "    JOIN sys.database_principals pr ON dp.grantee_principal_id = pr.principal_id " +
18         "    WHERE pr.name = SUSER_SNAME() AND dp.permission_name = 'UNMASK') THEN 1 " +
19         "  ELSE 0 END", cn);
20
21     bool esAdministrador = Convert.ToInt32(cmd.ExecuteScalar()) == 1;
22     Conexion.IniciarSesion(usuario, contrasena, esAdministrador);
23     DialogResult = DialogResult.OK;
24     Close();
25 }

```

```

1 // Form1.cs - metodo reutilizable que ejecuta una consulta y la vuelca en la grilla
2 private void MostrarTabla(string consulta)
3 {
4     using SqlConnection conexion = Conexion.Crear();
5     SqlDataAdapter adaptador = new SqlDataAdapter(consulta, conexion);
6     DataTable tabla = new DataTable();
7     adaptador.Fill(tabla);
8     dgvDatos.DataSource = tabla;
9 }

```



```

10
11 // Boton "Resumen (Cursor)": ejecuta sp_ResumenSeguridadPorAnalista
12 private void btnResumen_Click(object? sender, EventArgs e)
13 {
14     using SqlConnection conexion = Conexion.Crear();
15     using SqlCommand comando = new SqlCommand("dbo.sp_ResumenSeguridadPorAnalista", conexion);
16     comando.CommandType = CommandType.StoredProcedure;
17
18     SqlDataAdapter adaptador = new SqlDataAdapter(comando);
19     DataTable tabla = new DataTable();
20     adaptador.Fill(tabla);
21     dgvDatos.DataSource = tabla;
22 }

```

6.2. Página web

Al enterarnos de que otros grupos estaban complementando su proyecto con una aplicación web, se decidió construir un complemento en **ASP.NET Core Razor Pages** (sigue siendo C# y Visual Studio) para que el sistema se viera más completo. A diferencia de la aplicación de escritorio -donde solo una persona la usa a la vez y el estado de sesión puede guardarse en una simple variable-, en una aplicación web varias personas pueden conectarse *al mismo tiempo* con usuarios distintos; por ello, cada dato de sesión se guarda en la **sesión de navegador** de cada visitante (ASP.NET Core Session) en vez de una variable compartida del servidor. Deliberadamente, la web se limitó a un alcance de **solo lectura** (no permite registrar ni cerrar incidentes).

```

1 // SessionExtensions.cs - guarda los datos de sesion por cada visitante, no en el servidor
2 public static class SessionExtensions
3 {
4     private const string ClaveCadena = "CadenaConexion";
5     private const string ClaveUsuario = "NombreUsuario";
6     private const string ClaveAdmin = "EsAdministrador";
7
8     public static void IniciarSesion(this ISession session, string cadena, string usuario, bool esAdmin)
9     {
10         session.SetString(ClaveCadena, cadena);
11         session.SetString(ClaveUsuario, usuario);
12         session.SetString(ClaveAdmin, esAdmin ? "1" : "0");
13     }
14
15     public static string ObtenerCadenaConexion(this ISession session) => session.GetString(ClaveCadena) ??
16         "";
17     public static bool ObtenerEsAdministrador(this ISession session) => session.GetString(ClaveAdmin) == "1";
18     public static bool EstaLogueado(this ISession session) => !string.IsNullOrEmpty(session.GetString(ClaveCadena));
19 }

```

```

1 // Login.cshtml.cs - mismo mecanismo de autenticacion que la app de escritorio
2 public IActionResult OnPost()
3 {
4     string cadena = ConexionBuilder.Construir(Usuario.Trim(), Contraseña);
5     using SqlConnection cn = new SqlConnection(cadena);
6     cn.Open();
7
8     using SqlCommand cmd = new SqlCommand(
9         "SELECT CASE " +
10         " WHEN IS_ROLEMEMBER('db_owner') = 1 THEN 1 " +
11         " WHEN EXISTS (SELECT 1 FROM sys.database_permissions dp " +
12         " JOIN sys.database_principals pr ON dp.grantee_principal_id = pr.principal_id " +
13         " WHERE pr.name = SUSER_SNAME() AND dp.permission_name = 'UNMASK') THEN 1 " +
14         " ELSE 0 END", cn);
15
16     bool esAdministrador = Convert.ToInt32(cmd.ExecuteScalar()) == 1;
17     HttpContext.Session.IniciarSesion(cadena, Usuario.Trim(), esAdministrador);
18     return RedirectToPage("/Index");
19 }

```

```

1 // Analistas.cshtml.cs - el enmascaramiento se aplica solo, sin logica adicional aqui
2 public IActionResult OnGet()
3 {
4     IActionResult? redir = RedirigirSiNoHayLogin();
5     if (redir != null) return redir;
6
7     using SqlConnection conexion = new SqlConnection(CadenaConexion);
8     string consulta = "SELECT id_analista, nombre, apellido, cargo, especializacion, " +
9                       "nivel_acceso, email, telefono, id_organizacion FROM dbo.ANALISTA " +
10                      "ORDER BY id_analista";
11
12     SqlDataAdapter adaptador = new SqlDataAdapter(consulta, conexion);
13     adaptador.Fill(Tabla);
14     return Page();
15 }

```

7. Hardening

El **hardening** (endurecimiento) se aplicó porque un sistema de gestión de incidentes de seguridad debe, ante todo, predicar con el ejemplo: no tendría sentido construir una plataforma para monitorear la seguridad de terceros si la propia base de datos no protege sus datos sensibles. Se implementaron tres técnicas, cada una resolviendo un problema concreto:

- **Principio de menor privilegio:** cada persona debe tener acceso únicamente a lo estrictamente necesario para su rol. Se crearon logins nominales con dos perfiles: un *administrador* y varios *clientes* de solo consulta.
- **Enmascaramiento Dinámico de Datos (DDM):** protege información sensible (datos de contacto del personal interno, huella técnica de infraestructura, detalles de vulnerabilidades sin parchear) para que solo el administrador la vea en claro.
- **Auditoría de accesos:** deja registro de cada inicio de sesión, correcto o fallido, sobre el servidor.

7.1. Menor privilegio: logins nominales

```

1 -- Logins a nivel de servidor (CHECK_POLICY=OFF: contraseñas simples solo de laboratorio)
2 CREATE LOGIN [Jose Escajadillo] WITH PASSWORD = '7ycabhcnhej', CHECK_POLICY = OFF;
3 CREATE LOGIN [Fabrizio Valencia] WITH PASSWORD = 'fabrizio123', CHECK_POLICY = OFF;
4 CREATE LOGIN [Diego Borja] WITH PASSWORD = 'diegoxyz', CHECK_POLICY = OFF;
5 CREATE LOGIN [Pedro Carpio] WITH PASSWORD = 'basededatos', CHECK_POLICY = OFF;
6 GO
7
8 USE SGISI;
9 GO
10 CREATE USER [Jose Escajadillo] FOR LOGIN [Jose Escajadillo];
11 CREATE USER [Fabrizio Valencia] FOR LOGIN [Fabrizio Valencia];
12 CREATE USER [Diego Borja] FOR LOGIN [Diego Borja];
13 CREATE USER [Pedro Carpio] FOR LOGIN [Pedro Carpio];
14 GO
15
16 -- Los 4 pueden leer y ejecutar procedimientos/funciones
17 ALTER ROLE db_datareader ADD MEMBER [Jose Escajadillo];
18 ALTER ROLE db_datareader ADD MEMBER [Fabrizio Valencia];
19 ALTER ROLE db_datareader ADD MEMBER [Diego Borja];
20 ALTER ROLE db_datareader ADD MEMBER [Pedro Carpio];
21 GO
22 GRANT EXECUTE TO [Jose Escajadillo], [Fabrizio Valencia], [Diego Borja], [Pedro Carpio];
23 GO
24
25 -- Solo el administrador puede escribir (registrar/cerrar incidentes)
26 ALTER ROLE db_datawriter ADD MEMBER [Jose Escajadillo];
27 GO

```

```

28
29 -- A los clientes se les niega explicitamente la escritura, aunque tengan EXECUTE general
30 DENY EXECUTE ON dbo.sp_RegistrarIncidente TO [Fabrizio Valencia], [Diego Borja], [Pedro Carpio];
31 DENY EXECUTE ON dbo.sp_CerrarIncidente TO [Fabrizio Valencia], [Diego Borja], [Pedro Carpio];
32 GO

```

7.2. Enmascaramiento Dinámico de Datos (DDM)

```

1 USE SGISI;
2 GO
3
4 -- ANALISTA: datos de contacto del personal interno de la consultora
5 ALTER TABLE dbo.ANALISTA
6     ALTER COLUMN email VARCHAR(150) MASKED WITH (FUNCTION = 'email()') NOT NULL;
7 ALTER TABLE dbo.ANALISTA
8     ALTER COLUMN telefono VARCHAR(20) MASKED WITH (FUNCTION = 'partial(0,"XXXXXX",4)') NULL;
9 ALTER TABLE dbo.ANALISTA
10    ALTER COLUMN nivel_acceso TINYINT MASKED WITH (FUNCTION = 'default()') NOT NULL;
11 GO
12
13 -- ACTIVO: huella tecnica de infraestructura (IP y SO exactos)
14 ALTER TABLE dbo.ACTIVO
15     ALTER COLUMN direccion_ip VARCHAR(45) MASKED WITH (FUNCTION = 'default()') NULL;
16 ALTER TABLE dbo.ACTIVO
17     ALTER COLUMN sistema_operativo VARCHAR(80) MASKED WITH (FUNCTION = 'default()') NULL;
18 GO
19
20 -- VULNERABILIDAD: detalle exacto de como es vulnerable un activo
21 ALTER TABLE dbo.VULNERABILIDAD
22     ALTER COLUMN cve_id VARCHAR(30) MASKED WITH (FUNCTION = 'default()') NULL;
23 ALTER TABLE dbo.VULNERABILIDAD
24     ALTER COLUMN descripcion VARCHAR(500) MASKED WITH (FUNCTION = 'default()') NOT NULL;
25 GO
26
27 -- SUPERVISOR / SUPERVISADO: jerarquia interna del personal
28 ALTER TABLE dbo.SUPERVISOR
29     ALTER COLUMN num_analistas_cargo INT MASKED WITH (FUNCTION = 'default()') NULL;
30 ALTER TABLE dbo.SUPERVISADO
31     ALTER COLUMN nivel_experiencia VARCHAR(20) MASKED WITH (FUNCTION = 'default()') NOT NULL;
32 ALTER TABLE dbo.SUPERVISADO
33     ALTER COLUMN area_operacion VARCHAR(100) MASKED WITH (FUNCTION = 'default()') NOT NULL;
34 GO
35
36 -- Solo el administrador (Jose Escajadillo) ve los datos reales
37 GRANT UNMASK ON dbo.ANALISTA(email) TO [Jose Escajadillo];
38 GRANT UNMASK ON dbo.ANALISTA(telefono) TO [Jose Escajadillo];
39 GRANT UNMASK ON dbo.ANALISTA(nivel_acceso) TO [Jose Escajadillo];
40 GRANT UNMASK ON dbo.ACTIVO(direccion_ip) TO [Jose Escajadillo];
41 GRANT UNMASK ON dbo.ACTIVO(sistema_operativo) TO [Jose Escajadillo];
42 GRANT UNMASK ON dbo.VULNERABILIDAD(cve_id) TO [Jose Escajadillo];
43 GRANT UNMASK ON dbo.VULNERABILIDAD(descripcion) TO [Jose Escajadillo];
44 GRANT UNMASK ON dbo.SUPERVISOR(num_analistas_cargo) TO [Jose Escajadillo];
45 GRANT UNMASK ON dbo.SUPERVISADO(nivel_experiencia) TO [Jose Escajadillo];
46 GRANT UNMASK ON dbo.SUPERVISADO(area_operacion) TO [Jose Escajadillo];
47 GO

```

7.3. Auditoría de accesos

```

1 USE master;
2 GO
3
4 -- Auditoria a nivel de servidor, filtrada a los logins de SGISI
5 CREATE SERVER AUDIT AUDIT_SGISI
6 TO FILE (FILEPATH = 'D:\SQL_AUDILOG_SGISI', MAXSIZE = 10 MB, MAX_ROLLOVER_FILES = 10)
7 WHERE server_principal_name = 'Jose Escajadillo'
8     OR server_principal_name = 'Fabrizio Valencia'
9     OR server_principal_name = 'Diego Borja'
10    OR server_principal_name = 'Pedro Carpio';
11 GO

```

```
12 ALTER SERVER AUDIT AUDIT_SGISI WITH (STATE = ON);
13 GO
14
15 CREATE SERVER AUDIT SPECIFICATION Audit_SGISI_Logins
16 FOR SERVER AUDIT AUDIT_SGISI
17     ADD (FAILED_LOGIN_GROUP),
18     ADD (SUCCESSFUL_LOGIN_GROUP);
19 GO
20 ALTER SERVER AUDIT SPECIFICATION Audit_SGISI_Logins WITH (STATE = ON);
21 GO
22
23 -- Permiso puntual para que la app pueda leer su propio log sin ser sysadmin
24 GRANT VIEW SERVER SECURITY AUDIT TO [Jose Escajadillo];
25 GO
26
27 -- Lectura del log de auditoria
28 SELECT event_time, action_id, succeeded, server_principal_name, host_name
29 FROM sys.fn_get_audit_file('D:\SQL_AUDILOG_SGISI\*', DEFAULT, DEFAULT)
30 ORDER BY event_time DESC;
```